

Starter Pattern

```
1 #include <stdio.h> // printf, scanf, puts, fgets
2 #include <string.h> // strlen, strcmp, strcpy, strchr...
3 #include <stdlib.h> // malloc, free, atoi, strtol
4 #include <ctype.h> // isalpha, isdigit, toupper...
5 #include <math.h> // sin, cos, pow, sqrt, M_PI
6
7 int main(void) {
8     printf("Hello, C!\n");
9     return 0;
10 }
```

Variables

```
1 // Declaration + initialization
2 int i = 42;
3 double pi = 3.14159;
4 char c = 'A'; // character
5 char s[32] = "hi"; // char array (string)
```

Lvalues vs rvalues: left side must be modifiable storage.

Const: `const int n = 10;` (read-only).

Common Operators

Arithmetic:	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>	Assign:
	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code>	
Compare:	<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>	Logical: <code>&&</code> <code>—</code>
	<code>!—</code> (short-circuit)	
Bitwise:	<code>&</code> <code><<</code> <code>>></code> <code> </code> Addr/DeRef: <code>&</code> <code>*</code>	

Operator Precedence (high→low, excerpt)

`()` → unary `+` `-` `!` `~` `*` `&` `++` `--` → `*` `/` `%` → `+` `-` → shifts `<<` `>>` → comparisons `<` `<=` `>` `>=` → `==` `!=` → `&` `^` `—` → `&&` `—` → `||` → assignments.

Control Flow

if / else

```
1 if (x > 0) { puts("pos"); }
2 else if (x == 0) { puts("zero"); }
3 else { puts("neg"); }
```

for / while / do-while

```
1 // for: init; condition; step
2 for (int k = 0; k < 5; k++) { printf("%d ", k); }
3
4 int n = 5;
5 while (n > 0) { n--; } // check then run
6
7 int m = 0;
8 do { m++; } while (m < 3); // run then check
```

switch

```
1 switch (ch) {
2     case 'y': case 'Y': puts("yes"); break;
3     case 'n': case 'N': puts("no"); break;
4     default: puts("huh?");
5 }
```

break / continue / return

```
1 for (int i=0; i<10; i++){
2     if (i==5) break; // leave loop
3     if (i%2) continue; // skip odds
4     if (i==4) return 0; // leave func
5 }
```

Arrays

```
1 int a[4] = {1,2,3,4};
2 int b[4] = {0}; // zero-initialize
3 size_t len = sizeof a / sizeof a[0];
4 for (size_t i=0; i<len; i++) printf("%d ", a[i]);
```

Arrays **do not** self-know their length; pass size to functions.

Pointers

```
1 int x = 7;
2 int *p = &x; // p holds address of x
3 *p = 9; // writes to x via p
```

Pointer arithmetic is in units of the pointed-to type: `p+1` jumps `sizeof *p` bytes.

I/O & Formatting

printf (output)

Integers: `%d` `%i` `%u` `%x` `%o`
Width/precision: `%8d`, `%08x`, `%.3f`, `%8.2f`
Flags: `-` `+` `0` `#` e.g., `%+d`, `%#x`
Length: `%ld` `%lld` `%zu` `%td` `%jd` (long, long long, `size_t`, `ptrdiff_t`, `intmax_t`)
Floating: `%f` `%e` `%g` `%a` **Pointer:** `%p` **String:** `%s` **Char:** `%c`

```
1 printf("i=%d hex=%#x pi=%.2f s='%s'\n", 26, 26,
2       3.14159, "hi");
```

scanf (input)

Always pass addresses! `&i`, `&d`.
double uses `%lf`; **float** requires `%f` with `scanf`.
Use the return value (count of fields read).

```

1 int i; double d;
2 if (scanf("%d %lf", &i, &d) == 2) {
3     printf("ok: %d %.3f\n", i, d);
4 }

```

Reading lines safely

```

1 char buf[64];
2 if (fgets(buf, sizeof buf, stdin)) {
3     // buf may include '\n'; strip if you need to
4 }

```

Escape Sequences

\n newline, \t tab, \\ backslash, \" quote, \0 null.

Strings (char arrays)

```

1 char s[16] = "cat";           // 'c' 'a' 't' '\0'
2 char t[16]; strcpy(t, s);    // beware overflow
3 strncat(s, "nip", sizeof s - strlen(s) - 1);
4 size_t n = strlen(s);        // length w/o '\0'

```

Safer building: snprintf(dst, size, "x=%d", x);
Adjacent string literals concatenate at compile time:

```

1 printf("ab" "cd"); // prints "abcd"

```

Common string.h

strlen(s)	length excluding '\0'
strcmp(a,b)	¡0, 0, ¡0 lexicographic compare
strcpy dst,src	copy (unsafe if too small)
strncpy	size-limited (watch for no ")
strcat/strncat	append (watch sizes)
strchr(s,'x')	first occurrence or NULL
strstr(s,"sub")	substring or NULL

Expressions & Evaluation

Integers & floats

5/2 → **2** (int division, trunc toward 0)
5/2.0 → **2.5** 7%4 → **3**
For C99+: remainder has sign of dividend: -7%4 → **-3**.

Chars (ASCII)

'A' → 65 'B' == 'A'+1 → **1 (true)**
'0' → 48 '0' + 7 → **55** (character '7')
'a' - 'A' → **32**

Truthiness

Nonzero is **true**. !0 == 1, !5 == 0.
a && b stops if a==0; a — b— stops if a!=0.

Ternary

cond ? A : B picks A if true, else B. Example:
int sign = (x>=0) ? +1 : -1;

String literal facts

- "hi" has type const char[3]: {'h','i','\0'}.
- "ab" "cd" ⇒ "abcd" at compile time.
- "a" + "b" is **invalid** (no string addition in C).

Functions

Declaring/defining

```

1 int add(int x, int y);           // prototype
2 int add(int x, int y){ return x+y; }
3
4 static int sqr(int x){ return x*x; }
5 // internal linkage

```

Passing arrays/pointers

```

1 void fill(int *a, size_t n, int v){
2     for (size_t i=0;i<n;i++) a[i]=v;
3 }
4 int data[8]; fill(data, 8, 7);

```

Common library picks

ctype.h	isalpha, isdigit, isspace, toupper, tolower
math.h	sqrt, pow, sin, cos, fabs, floor, ceil
stdlib.h	atoi, strtol, rand, qsort, bsearch, exit
stdio.h	printf, scanf, puts, fgets, fopen, fclose

Memory (basics)

```

1 int *arr = malloc(10 * sizeof *arr);
2 if (!arr) { /* handle alloc failure */ }
3 free(arr);

```

Rule: every successful malloc/calloc/realloc must be paired with free exactly once.

Idioms & Patterns

```

1 // iterate over a C-string
2 for (char *p = s; *p; ++p) { putchar(*p); }
3
4 // parse int with error-checking
5 char *end; long v = strtol(s, &end, 10);
6 if (end==s) { /* no digits */ }
7
8 // bounded print into buffer
9 char buf[64];
10 snprintf(buf, sizeof buf, "x=%d", x);
11
12 // min/max macros
13 #define MIN(a,b) ((a)<(b)?(a):(b))
14 #define MAX(a,b) ((a)>(b)?(a):(b))

```